Task 1

Applying SRP (Online Learning Platform Scenario)

1. Explain why this violates the Single Responsibility Principle.

Currently, one single class handles all the responsibilities of the platform. This violates SRP because the class has multiple reasons to change. For example, if the platform decides to switch from email to SMS notifications, the developer must modify the same class that handles student enrollment and progress tracking, risking bugs in unrelated features.

2. Identify at least three separate responsibilities.

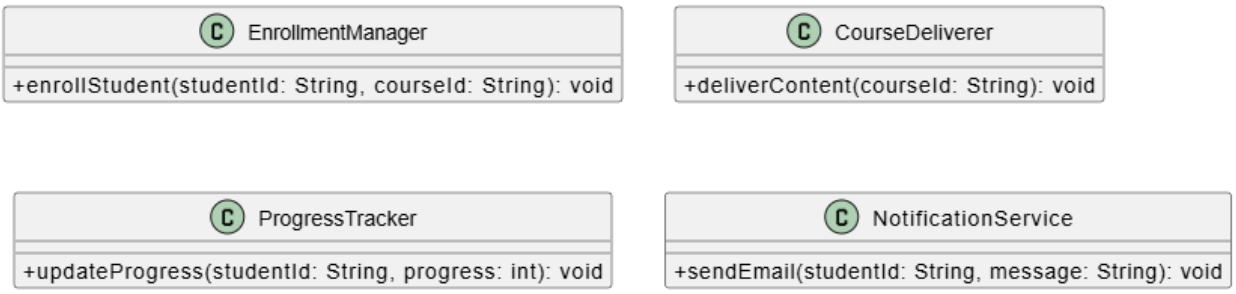
Based on the scenario, we can extract the following distinct responsibilities:

- Student enrollment management.
- Course content delivery.
- Progress tracking.
- Email notifications to students.

3. Redesign the system by separating responsibilities into different classes.

We will separate these responsibilities into four distinct classes: EnrollmentManager, CourseDeliverer, ProgressTracker, and NotificationService. This way, a change to the notification system will only affect NotificationService.

4. Draw a UMLclass diagram of your improved design.



5. Provide sample code for your design.

```
#include <iostream>
#include <string>

using namespace std;

class EnrollmentManager {
public:
    void enrollStudent(string studentId, string courseId) {
        cout << "Enrolling student " << studentId << " in course " <<
courseId << endl;
    }
};

class CourseDeliverer {
public:
    void deliverContent(string courseId) {
        cout << "Delivering content for course " << courseId << endl;
    }
};

class ProgressTracker {
public:
    void updateProgress(string studentId, int progress) {
        cout << "Updating progress for " << studentId << " to " << progress
<< "%" << endl;
    }
};

class NotificationService {
public:
    void sendEmail(string studentId, string message) {
        cout << "Sending email to " << studentId << ": " << message << endl;
    }
};

int main() {
    EnrollmentManager enrollment;
    ProgressTracker progress;
```

```
    NotificationService notification;

    enrollment.enrollStudent("S101", "CS101");
    progress.updateProgress("S101", 50);
    notification.sendEmail("S101", "You have completed 50% of your course!");

    return 0;
}
```

Task 2

Applying OCP (Movie Streaming Service Scenario)

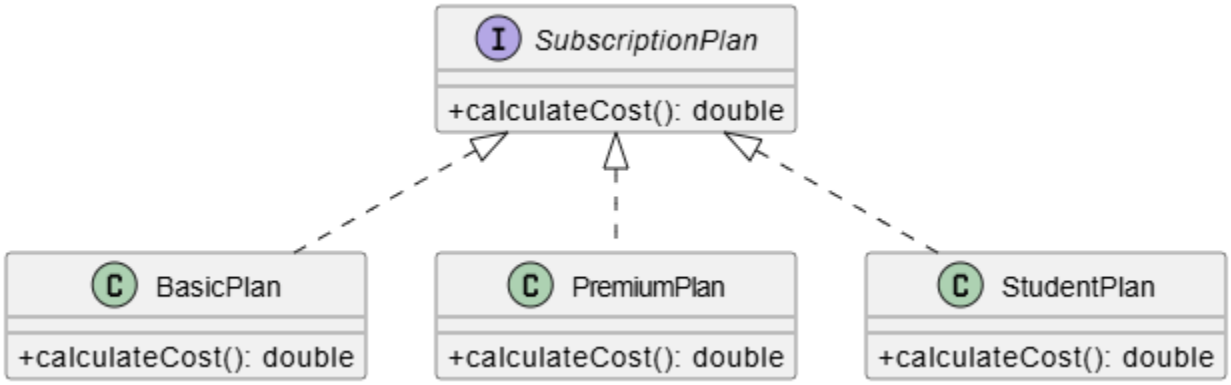
1. Explain why modifying the same class repeatedly is risky.

The current pricing class must be edited every time a new plan is added. Modifying existing code is risky because it can introduce bugs that break the old logic for the Basic Plan and Premium Plan. It makes testing take longer and the class becomes fragile.

2. Redesign the system using OCP.

To follow the Open/Closed Principle, we should make the system open for extension but closed for modification. We will create a SubscriptionPlan interface. Any new plan (like Family, Student, or Ad-supported) will simply implement this interface without requiring us to touch the existing code.

3. Draw a UMLdiagram showing how new plans can be added easily.



4. Write sample code supporting extensibility.

```cpp
#include <iostream>

using namespace std;

// The Abstraction (using a pure virtual function to act as an interface)
class SubscriptionPlan {
public:
    virtual double calculateCost() const = 0;
    virtual ~SubscriptionPlan() = default; // Good practice for polymorphic
base classes
};

// Existing Plans
class BasicPlan : public SubscriptionPlan {
public:
    double calculateCost() const override {
        return 9.99;
    }
};

class PremiumPlan : public SubscriptionPlan {
public:
    double calculateCost() const override {
        return 19.99;
    }
};

// Easily extending the system with new plans without modifying old code
class StudentPlan : public SubscriptionPlan {
public:
    double calculateCost() const override {
        return 4.99;
```

```
    }
};

class FamilyPlan : public SubscriptionPlan {
public:
    double calculateCost() const override {
        return 24.99;
    }
};

int main() {
    SubscriptionPlan* myPlan = new StudentPlan();
    cout << "Cost of Student Plan: $" << myPlan->calculateCost() << endl;

    delete myPlan;
    return 0;
}
```

Task 3

Applying LSP (Gaming System Scenario)

1. Explain howforcing Healer to implement attack() violates LSP.

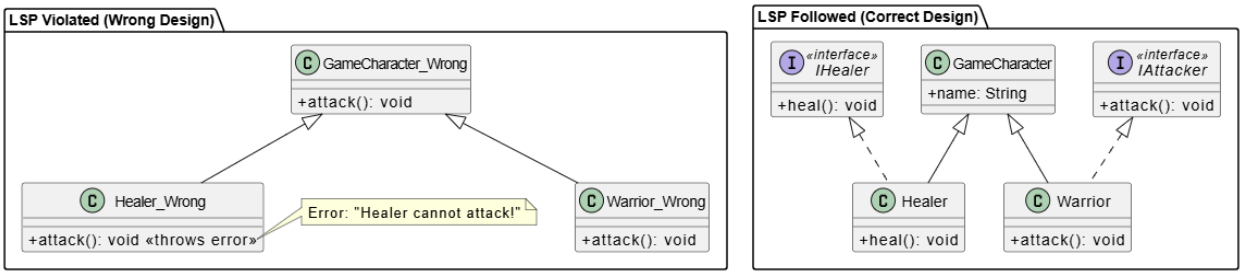
Forcing a Healer to implement an attack() method violates the Liskov Substitution Principle because a subclass must work in place of its parent class without errors. If a game loop treats all objects as GameCharacter and calls attack() on them, a Healer object would likely throw an exception or produce incorrect behavior. The Healer fails to fulfill the behavioral expectations set by the base GameCharacter class.

2.Redesign the hierarchy properly.

Not all characters attack. We should remove the attack() method from the base GameCharacter class . Instead, we can create separate, behavior-specific interfaces such as IAttacker and IHealer. A Warrior

will inherit from GameCharacter and implement IAttacker, while a Healer will inherit from GameCharacter and implement IHealer .

3.DrawUMLdiagrams before and after redesign.



4. Provide corrected code.

```
#include <iostream>
#include <string>

using namespace std;

// Base class with common character properties
class GameCharacter {
protected:
    string name;
public:
    GameCharacter(string n) : name(n) {}
    virtual ~GameCharacter() = default;
    string getName() const { return name; }
};

// Behavior-specific interfaces
class IAttacker {
public:
    virtual void attack() = 0;
    virtual ~IAttacker() = default;
};

class IHealer {
public:
    virtual void heal() = 0;
    virtual ~IHealer() = default;
};

// Concrete Classes
class Warrior : public GameCharacter, public IAttacker {
public:
    Warrior(string n) : GameCharacter(n) {}

    void attack() override {
```

```
        cout << name << " swings a broadsword and deals 50 damage!" << endl;
    }
};

class Healer : public GameCharacter, public IHealer {
public:
    Healer(string n) : GameCharacter(n) {}

    void heal() override {
        cout << name << " casts a restoration spell and restores 30 HP!" <<
endl;
    }
};

int main() {
    Warrior myWarrior("Thor");
    Healer myHealer("Mercy");

    myWarrior.attack();
    myHealer.heal();

    // You can safely use IAttacker pointers for entities guaranteed to
attack
    IAttacker* attacker = &myWarrior;
    attacker->attack();

    return 0;
}
```

Task 4

Applying ISP (Smart Home System Scenario)

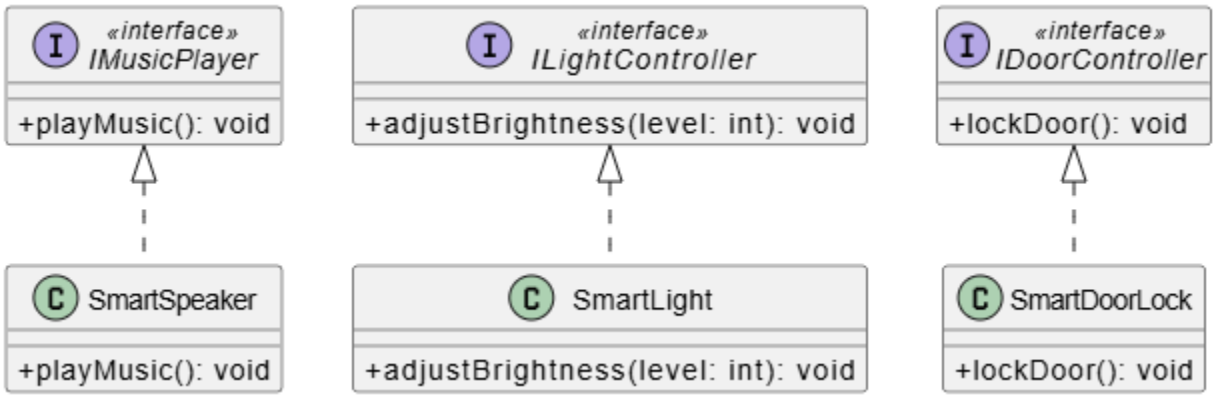
1. Explain why this violates ISP.

The Interface Segregation Principle states that clients shouldn't be forced to depend on methods they do not use. A large, "fat" interface containing playMusic(), adjustBrightness(), and lockDoor() forces devices like Smart Lights to implement dummy or exception-throwing methods for music and door locking. This makes the code ugly, confusing, and error-prone

2. Split the interface into smaller ones.

We should break the single large interface down into smaller, highly specific interfaces based on capability . We will create IMusicPlayer, ILightController, and IDoorController. Each smart device will then only inherit the interface that matches its actual functionality.

3. Draw UMLdiagrams.



4. Provide example implementation.

```cpp
#include <iostream>

using namespace std;

// Segregated Interfaces
class IMusicPlayer {
public:
    virtual void playMusic() = 0;
    virtual ~IMusicPlayer() = default;
};

class ILightController {
public:
    virtual void adjustBrightness(int level) = 0;
    virtual ~ILightController() = default;
};

class IDoorController {
public:
    virtual void lockDoor() = 0;
    virtual ~IDoorController() = default;
};
```

```
// Concrete Device Classes implementing only what they need
class SmartLight : public ILightController {
public:
    void adjustBrightness(int level) override {
        cout << "[Smart Light] Brightness adjusted to " << level << "%" <<
endl;
    }
};

class SmartSpeaker : public IMusicPlayer {
public:
    void playMusic() override {
        cout << "[Smart Speaker] Playing your favorite playlist..." << endl;
    }
};

class SmartDoorLock : public IDoorController {
public:
    void lockDoor() override {
        cout << "[Smart Door Lock] The front door is now securely locked." <<
endl;
    }
};

int main() {
    SmartLight livingRoomLight;
    SmartSpeaker echoDot;
    SmartDoorLock frontDoor;

    livingRoomLight.adjustBrightness(75);
    echoDot.playMusic();
    frontDoor.lockDoor();

    return 0;
}
```

Task 5

Applying DIP (E-Library System Scenario)

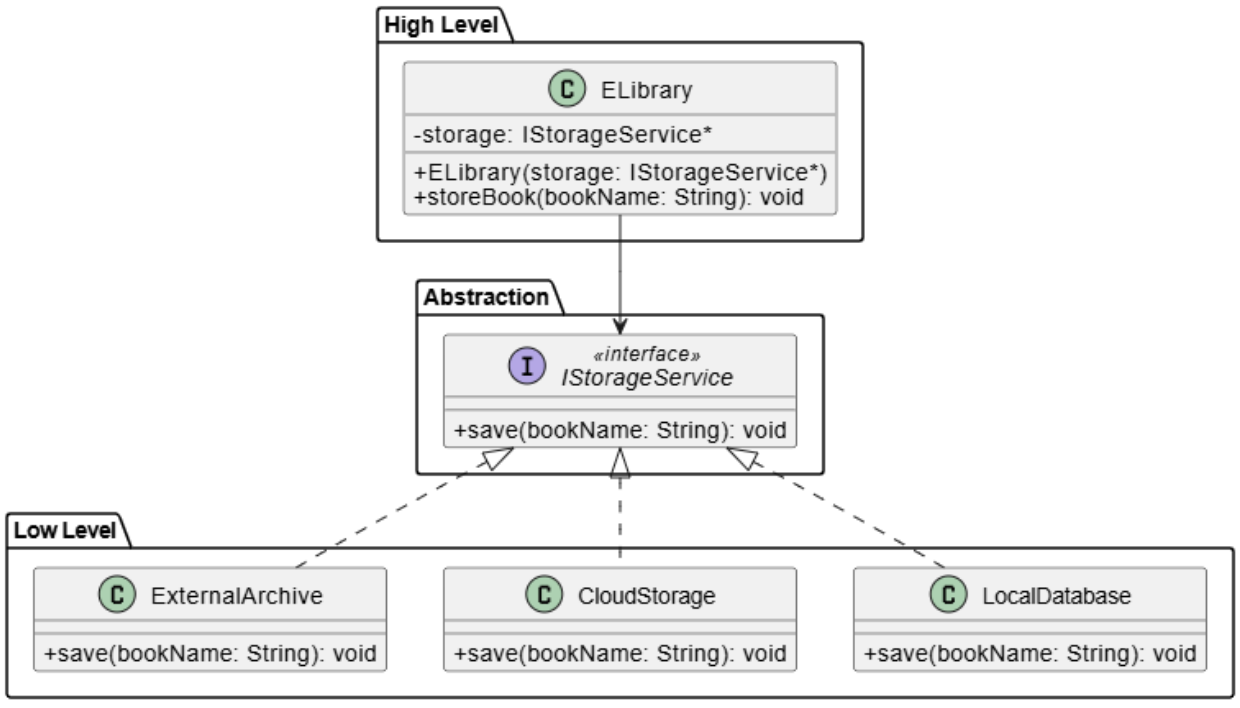
1. Explain why direct dependency is problematic.

Currently, the high-level E-Library system depends directly on a low-level implementation detail (`LocalDatabase`). This is problematic because any changes to how data is stored—such as moving to cloud storage or an external digital archive—will require modifying the core E-Library business logic. This tightly couples the system and violates the Dependency Inversion Principle, which states that high-level classes should depend on abstractions, not low-level classes .

2. Redesign using DIP.

To fix this, we need to introduce an abstraction (an interface). We will create an `IStorageService` interface. The `LocalDatabase`, `CloudStorage`, and `ExternalArchive` classes will all implement this interface. The `ELibrary` class will then depend only on the `IStorageService` abstraction, typically passed in via its constructor (dependency injection).

3. Draw UMLshowing abstraction usage.



4. Provide sample flexible code.

```
#include <iostream>
#include <string>

using namespace std;

// The Abstraction
class IStorageService {
public:
    virtual void save(string bookName) = 0;
    virtual ~IStorageService() = default;
};

// Low-level implementations
class LocalDatabase : public IStorageService {
public:
    void save(string bookName) override {
        cout << "[Local DB] Saving '" << bookName << "' to the local hard
drive." << endl;
    }
};

class CloudStorage : public IStorageService {
public:
    void save(string bookName) override {
        cout << "[Cloud Storage] Uploading '" << bookName << "' to the cloud
server." << endl;
    }
};

class ExternalArchive : public IStorageService {
public:
    void save(string bookName) override {
        cout << "[External Archive] Archiving '" << bookName << "' to the
global digital library." << endl;
    }
};

// High-level class
class ELibrary {
private:
    IStorageService* storage; // Depends on abstraction, not concrete class
public:
    // Constructor Injection
    ELibrary(IStorageService* s) : storage(s) {}

    void storeBook(string bookName) {
        cout << "E-Library is processing a new book..." << endl;
        storage->save(bookName);
    }

    // Allow dynamic switching of the storage strategy at runtime
    void setStorageService(IStorageService* s) {
        storage = s;
    }
```

```
};

int main() {
    LocalDatabase localDb;
    CloudStorage cloudDb;

    // Start with local database
    ELibrary myLibrary(&localDb);
    myLibrary.storeBook("Introduction to Algorithms");

    // Later, switch to cloud storage without changing ELibrary logic
    cout << "\n-- Switching to Cloud Storage --\n";
    myLibrary.setStorageService(&cloudDb);
    myLibrary.storeBook("Design Patterns: Elements of Reusable Object-
Oriented Software");

    return 0;
}
```

Task 6

MiniCase Study(Integrated SOLID)

1. Identify where at least three SOLID principles apply.

In a Fitness Tracking App managing workouts, diets, notifications, and reports , several principles apply:

- **Single Responsibility Principle (SRP):** The app should not be a single monolithic class. Workout tracking, diet planning, progress reporting, and notifications are four distinct responsibilities that should be separated into their own classes.
- **Open/Closed Principle (OCP):** The notification system should be open for extension. We should be able to add new notification types (Push, SMS, Email) without modifying the core notification manager.
- **Dependency Inversion Principle (DIP):** The main application or the report generator should depend on abstractions (like an `IDatabase` or `INotificationService` interface) rather than concrete implementations, allowing the app to easily swap out databases or notification providers .

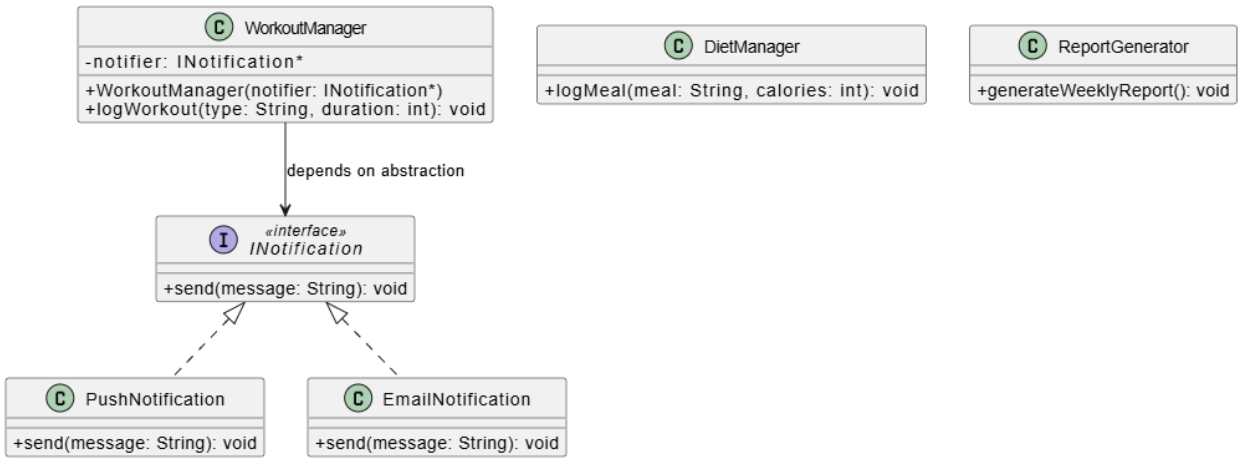
2. Suggest design improvements.

- Create separate managers: `WorkoutManager`, `DietManager`, and `ReportGenerator` (SRP).

- Create an `INotification` interface that `PushNotification`, `EmailNotification`, and `SMSNotification` implement (OCP).

- Inject the `INotification` dependency into the `WorkoutManager` so it can alert users without knowing exactly *how* the alert is sent (DIP).

3. Draw UMLdiagram.



4. Provide sample code for one improvement

```cpp
#include <iostream>
#include <string>

using namespace std;

// Abstraction for Notifications (DIP & OCP)
class INotification {
public:
    virtual void send(string message) = 0;
    virtual ~INotification() = default;
};

// Concrete Notification Types (OCP - easy to add more)
class PushNotification : public INotification {
public:
    void send(string message) override {
        cout << "[Push Alert to Phone] " << message << endl;
    }
};

class EmailNotification : public INotification {
public:
    void send(string message) override {
        cout << "[Email Sent] Subject: Fitness Update - " << message << endl;
    }
```

```
};

// Manager class handling only Workout Logic (SRP)
class WorkoutManager {
private:
    INotification* notifier; // Depends on abstraction (DIP)
public:
    WorkoutManager(INotification* n) : notifier(n) {}

    void logWorkout(string type, int durationMinutes) {
        cout << "Logging workout: " << type << " for " << durationMinutes <<
" mins." << endl;
        // Logic to save workout goes here...

        // Notify user
        notifier->send("Great job! You completed a " +
to_string(durationMinutes) + " min " + type + " workout.");
    }
};

int main() {
    PushNotification mobilePush;
    EmailNotification emailAlert;

    // User 1 prefers push notifications
    WorkoutManager user1Workout(&mobilePush);
    user1Workout.logWorkout("Running", 30);

    cout << "------------------------" << endl;

    // User 2 prefers email notifications
    WorkoutManager user2Workout(&emailAlert);
    user2Workout.logWorkout("Weightlifting", 45);

    return 0;
}
```